

Pypomp: Inference for partially observed Markov process models in Python with JAX DRAFT IN PROGRESS

Aaron J. Abkemeier¹

Jun Chen²

Kevin Tan³

Jesse Wheeler⁴

Bo Yang⁵

Kunyang He¹

Jonathan Terhorst¹

Aaron A. King

Edward L. Ionides¹

AARONABK@UMICH.EDU

CHENJUNC@UMICH.EDU

KEVTAN@WHARTON.UPENN.EDU

JESSEWHEELER@ISU.EDU

YBB@UMICH.EDU

KUNYANGHE@UMICH.EDU

JONTH@UMICH.EDU

IONIDES@UMICH.EDU

¹University of Michigan, Department of Statistics; ²Duke University, Department of Statistics;
³University of Pennsylvania, Department of Statistics; ⁴Idaho State University, Department of
Mathematics; ⁵Columbia University, Department of Marketing

Editor:

Abstract

Model development and parameter estimation for non-Gaussian partially observed Markov process (POMP) mechanistic models are fundamental challenges in a variety of fields including epidemiology, ecology, and finance. Particle filter methods can provide statistically efficient inference for highly nonlinear POMP models, but their practical application is limited by high computational demands. We introduce Pypomp, a high-performance Python library for statistical inference using POMP models. Pypomp includes state-of-the-art particle filtering algorithms to take advantage of GPU hardware, just-in-time computation, and automatic differentiation accessed using JAX. Basic inference algorithms run with throughput as high as 8000 times greater than on a CPU core, and advanced algorithms provide further acceleration. Pypomp provides a comprehensive interface that streamlines model construction, fitting, and analysis, allowing practitioners to develop and evaluate complex models with minimal implementation overhead.

Keywords: pomp, JAX, GPU acceleration, likelihood-based inference

1 Introduction

Partially observed Markov process (POMP) models, also known as state-space models or hidden Markov models, provide a flexible mechanistic framework for modeling dynamic systems whose underlying state is not directly observed. The particle filter (Arulampalam et al., 2002), also known as sequential Monte Carlo, estimates the likelihood of a POMP model, and iterated filtering (Ionides et al., 2006, 2015) extends it to maximum likelihood estimation. These methods only need to simulate from the process model, not evaluate its transition probabilities, a property known as plug-and-play (Bretó et al., 2009). Conse-

quently, they apply to a wide range of models unconstrained by mathematical convenience, and they are used in epidemiology (Mietchen et al., 2024; Fox et al., 2022; Wen et al., 2024), ecology (Auger-Méthé et al., 2021; Marino et al., 2019; Blackwood et al., 2013), finance (Bretó, 2014), and other domains where mechanistic dynamic modeling is desired.

Pypomp is a Python library combining advances in hardware, software, and algorithms to provide novel capabilities for POMP models, with an emphasis on inference via particle filters. It is built on JAX (Bradbury et al., 2018), which provides just-in-time (JIT) compilation, automatic differentiation (AD), and access to massively parallel hardware such as GPUs. Pypomp inherits its software design from the R package `pomp` (King et al., 2016) and implements the algorithmic advances needed to take advantage of JAX. For AD, it implements a differentiable particle filter that controls both bias and variance (Tan et al., 2024). Rejection sampling algorithms for common distributions are poorly suited to GPUs, so Pypomp also provides alternative samplers (Giles, 2016; Giles and Beentjes, 2024; Temme, 1992).

When the state space is small and discrete, or the model is linear Gaussian or close enough for Gaussian filtering approximations, the particle filter is unnecessary, and the Dynamax library (Linderman et al., 2025) provides a JAX-based solution. Pypomp is aimed at more complex, nonlinear models such as those common in ecology and epidemiology, although its plug-and-play property also makes it convenient for rapid model development when alternatives apply. It also implements panel methods (Bretó et al., 2020) that address the particle filter’s poor scaling with state dimension.

On a single GPU, Pypomp runs the iterated filtering 2 (IF2) algorithm 20 to 61 times faster than the R packages `pomp` and `panelPomp` on a 36-core CPU node, matching the throughput of 727 to 2,208 CPU cores running R (Section 4). This makes plug-and-play likelihood-based inference feasible on datasets that would otherwise be intractably large. In Section 4.1, we fit a panel POMP model to the England and Wales measles dataset of Korevaar et al. (2020), which contains over one million observations from 1,422 towns and cities. One GPU completed the fit in 29 hours, whereas we estimate that the same computation would take about 4.5 months with the R package `panelPomp` (Bretó et al., 2020).

[EL: A BRAINSTORMING IDEA: IN THE SMC READING GROUP, I STARTED USING THE PHRASE “MICHIGAN NOTATION” TO EXPLAIN OUR COHERENT SET OF NAMES AND NOTATIONS THAT IS SOMEWHAT DIFFERENT TO THE DIVERSE TERMINOLOGIES FOUND ELSEWHERE. I’M NOT SURE IF THIS IS THE RIGHT PLACE, BUT MAYBE IT IS. ANYWAY, HERE’S A GO]

2 POMP Modeling: The Michigan notation

Diverse terminologies and notations have evolved for POMP models in different communities. Pypomp uses the Michigan notation that arose from the program to build a unifying software environment to represent a wide range of proposed statistical methodologies in the R `pomp` package. Distinguishing features are: (i) t is used to represent continuous time, allowing proper handling of scientific units and situations where the latent variable is defined not just at times it is observed; (ii) $\{t_n, n = 1, \dots, N\}$ is a discrete sequence of N measurement times; (iii) ranges are upper-case Roman letters with the corresponding lower-case

often used as an index; (iv) joint and conditional densities are defined using appropriate subscripts and avoiding overloading, for example, $f_{Y_n|X_n}(y_n|x_n;\theta)$ denotes the conditional density of Y_n given X_n ; (v) sequences are denoted by colons, for example, $\{t_n, n = 1, \dots, N\}$ is written as $t_{1:N}$. [\[EL: TO BE CONTINUED, IF WE GO THIS ROUTE\]](#).

A POMP model consists of an unobserved Markov process $\{X(t), t_0 \leq t \leq t_N\}$ and observations $Y_{1:N}$ made at times $t_{1:N}$, with observed values $y_{1:N}^*$. The initial state is defined at time t_0 , $X_n := X(t_n)$, and Y_n is independent of Y_m, X_m , $m \neq n$, conditional on X_n . The model has a joint density $f_{Y_{1:N}, X_{0:N}}(y_{1:N}, x_{0:N}; \theta)$ with parameters θ , which may be intractable to evaluate. The particle filter estimates the likelihood $f_{Y_{1:N}}(y_{1:N}^*; \theta)$, and IF2 (Ionides et al., 2015) estimates the maximum likelihood estimate of θ , without evaluating this joint density or the transition density. For these plug-and-play methods, it is sufficient to specify:

- $X_0 \sim f_{X_0}(x_0; \theta)$: a simulator for the initial state.
- $X_{n+1} \sim f_{X_{n+1}|X_n}(x_{n+1}|x_n; \theta)$: a simulator for the state transition.
- $f_{Y_n|X_n}(y_n|x_n; \theta)$: the measurement density.

In Pypomp, the user passes the data and programmatic descriptions of these functions to a `Pomp` object.

3 Inference Methods in Pypomp

Pypomp implements the particle filter (`pfilter`) for likelihood evaluation and state estimation, IF2 (`mif`) for global maximum likelihood estimation, and the differentiable particle filter MOP- α (`mop`, Tan et al. (2024)), whose gradients the `train` method uses for local optimization. These are methods of the `Pomp` class, and each model component is a JAX-compatible function. For example, the S&P 500 model included in the package defines its state transition simulator as follows:

```
import jax
import jax.numpy as jnp
from pypomp.types import StateDict, ParamDict, CovarDict, TimeFloat, StepSizeFloat, RNGKey

def rproc(
    X_: StateDict, theta_: ParamDict, key: RNGKey, covars: CovarDict,
    t: TimeFloat, dt: StepSizeFloat
):
    V, S = X_["V"], X_["S"]
    mu, kappa, theta = theta_["mu"], theta_["kappa"], theta_["theta"]
    xi, rho, y_prev = theta_["xi"], theta_["rho"], covars["y_prev"]
    dZ = jax.random.normal(key)
    dWs = (y_prev - mu + 0.5 * V) / jnp.sqrt(V)
    dWv = rho * dWs + jnp.sqrt(1 - rho**2) * dZ
    S = S + S * (mu + jnp.sqrt(jnp.maximum(V, 1e-32)) * dWs)
```

```

V = V + kappa * (theta - V) + xi * jnp.sqrt(V) * dWv
V = jnp.maximum(V, 1e-32)
return {"V": V, "S": S}

```

Pypomp vectorizes such a function over particles and over replicate runs, such as when starting searches from many values of θ , and JIT compiles it so that the computation runs in parallel across GPU cores. The following example searches globally with IF2, refines the estimate by gradient descent, and evaluates the likelihood of the result:

```

import pypomp as pp
spx_obj = pp.models.spx()
key = jax.random.key(1)
rw_sd = pp.RWSigma( # Random walk standard deviations for IF2
    sigmas={"mu": 0.02, "kappa": 0.02, "theta": 0.02,
            "xi": 0.02, "rho": 0.02, "V_0": 0.1},
    init_names=["V_0"],
).geometric_cooling(a=0.5)
spx_obj.mif(rw_sd=rw_sd, J=1000, M=10, key=key) # IF2
eta = pp.LearningRate(
    {name: 0.001 for name in spx_obj.canonical_param_names}
).geometric_decay(decay_rate=0.5, M=10) # Learning rates
spx_obj.train(J=1000, M=10, eta=eta, optimizer=pp.Adam()) # Gradient descent
spx_obj.pfilter(J=1000, reps=10) # Particle filter

```

Each method call stores its log-likelihood estimates, parameter traces, execution time, and arguments in `spx_obj.results_history`, and helpers such as `traces` return these in convenient formats such as a data frame. The JIT-compiled functions behind these methods are available in `pypomp.functional` for building custom inference procedures. The source code is available at <https://github.com/pypomp/pypomp>, and documentation is available at <https://pypomp.readthedocs.io>.

4 Performance

We benchmarked Pypomp 1.0.5 against `pomp` 6.3 and `panelPomp` 1.7.0.0 using the quantitative test suite at <https://github.com/pypomp/quant> (commit ed62cee). Table 1 reports wall-clock times for the same workloads with R on a 36-core CPU node, Pypomp on the same node, and Pypomp on one RTX PRO 6000 Blackwell Server Edition GPU. R divided each workload evenly across its 36 cores, so one core would take 36 times as long. Pypomp on the CPU is faster than R for the cholera and S&P 500 models but slower for the London measles model, whose Poisson, binomial, and gamma draws are expensive for JAX on a CPU. The GPU times rely on the approximate inverse-CDF samplers in `pypomp.random`: with the standard `jax.random` samplers, IF2 for the London measles model took 29 times longer on the GPU.

Table 1: Wall-clock time in seconds for IF2 and for particle filter (PF) evaluation of the IF2 estimates. Each IF2 row runs 36 searches from the same starting points on every platform, and Pypomp PF times include JIT compilation. Speedup is the R time divided by the Pypomp GPU time. Throughput is the speedup times 36: the number of CPU cores running R that one GPU matches. Peak is the highest IF2 throughput in GPU scaling tests with more searches or more particles per search (Figure 1), projected to the benchmark’s number of IF2 iterations including JIT compilation. The 4-town panel has no PF comparison because the quant scripts filter different numbers of parameter sets, and it has no Pypomp CPU run.

Model	Task	Pypomp			Speedup	Throughput	Peak
		R (36 cores)	(36 cores)	Pypomp (1 GPU)		vs. 1 core	vs. 1 core
Cholera, Dhaka	IF2	1,566	390	28.8	54×	1,959×	3,152×
	PF	359	67.3	8.1	44×	1,590×	—
Measles, London	IF2	2,923	4,484	47.6	61×	2,208×	3,258×
	PF	895	1,544	19.5	46×	1,653×	—
Measles, 4-town panel	IF2	8,187	—	168	49×	1,759×	2,428×
	PF	67.4	—	75.4	—	—	—
S&P 500	IF2	703	275	34.8	20×	727×	8,659×
	PF	36.4	7.4	5.2	7×	251×	—

The S&P 500 model shows the smallest GPU advantage in Table 1 because its benchmark gives the GPU little parallel work: 36 searches of 1,000 particles, resampled after each of 3,523 cheap time steps, so fixed per-step costs dominate. Figure 1 shows that throughput grows with the number of particles filtered at once, and with 2,000 searches of 1,000 particles, one GPU matches 8,659 CPU cores running R. At a given total number of particles, adding searches beats adding particles per search by an average of only 6%, and peak GPU memory stays below 1 GB. For the cholera and measles models, whose benchmarks already give the GPU more work, the scaling tests raise IF2 throughput by a smaller factor, to between 2,428 and 3,258 CPU cores running R, 1.4 to 1.6 times the benchmark values (Table 1).

4.1 Fitting all towns in England and Wales

To show the scale of problem that this acceleration makes feasible, we fit the measles model of He et al. (2010) to weekly case reports from all 1,422 towns and cities in England and Wales between 1950 and 1963 (Korevaar et al., 2020), treating every parameter as specific to each town. We ran 12 panel IF2 searches, each with 10,000 particles for 200 iterations, and evaluated each endpoint with 36 particle filter replicates of 5,000 particles. We then combined the best estimate for each town across searches (Bretó et al., 2020) and evaluated

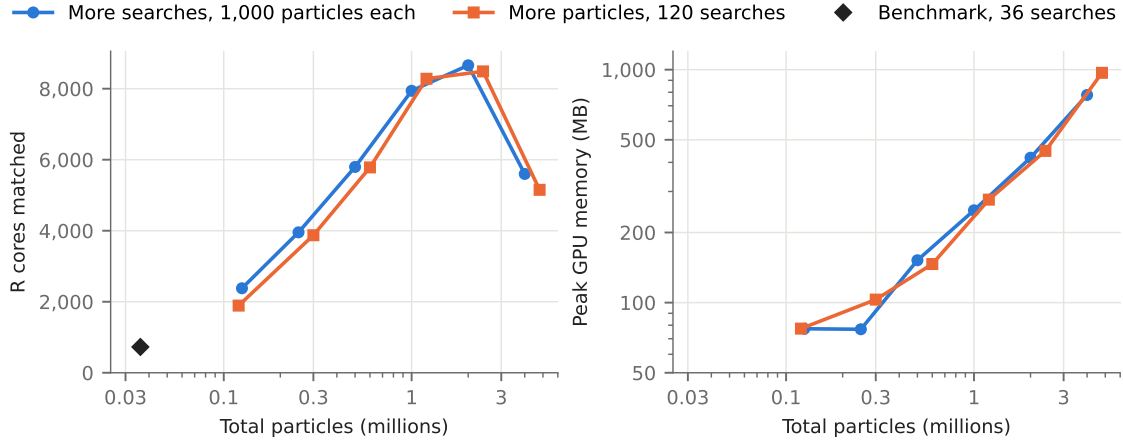


Figure 1: GPU throughput and memory for IF2 on the S&P 500 model as the total number of particles grows, either by adding searches of 1,000 particles or by adding particles to each of 120 searches. Left: CPU cores running R matched by one GPU, with the Table 1 benchmark in black. Right: peak GPU memory. Each configuration ran 10 IF2 iterations, projected to the benchmark’s 200 iterations including JIT compilation. All axes except the left y-axis use a log scale.

the result with another 36 replicates. Table 2 gives the time for each stage on one GPU with Pypomp 1.0.0.

For comparison, we estimated how long the same computation would take with `panelPomp` on a 36-core node. We measured `panelPomp`’s cost per particle, per IF2 iteration, and per town on the 4-town panel in Table 1, which covers the same period with the same daily time step, and scaled it linearly to the full computation. Each IF2 search is sequential, so R can use at most one core per search, and the extra cores only help with the particle filter evaluations. By this estimate, R would need about 4.5 months, 114 times longer than the GPU, and about 4.8 years on a single core, so the GPU delivered the throughput of about 1,439 CPU cores running R.

Table 2: Wall-clock time for fitting the measles model to all towns in England and Wales. Pypomp times were measured on one GPU. R times are estimates for `panelPomp` on a 36-core CPU node, extrapolated from Table 1.

Stage	Pypomp (1 GPU)	R <code>panelPomp</code> , 36 cores (estimated)
IF2	27.3 hours	134.7 days
PF, all search endpoints	99 minutes	3.3 days
PF, combined estimate	10 minutes	6.7 hours
Total	29.1 hours	138.3 days

5 Discussion

References

- M.S. Arulampalam, S. Maskell, N. Gordon, and T. Clapp. A tutorial on particle filters for online nonlinear/non-Gaussian Bayesian tracking. *IEEE Transactions on Signal Processing*, 50(2):174–188, February 2002. ISSN 1053587X. doi: 10.1109/78.978374.
- Marie Auger-Méthé, Ken Newman, Diana Cole, Fanny Empacher, Rowenna Gryba, Aaron A. King, Vianey Leos-Barajas, Joanna Mills Flemming, Anders Nielsen, Giovanni Petris, and Len Thomas. A guide to state-space modeling of ecological time series. *Ecological Monographs*, 91(4):e01470, 2021. doi: 10.1002/ecm.1470. URL <https://doi.org/10.1002/ecm.1470>.
- J. C. Blackwood, D. G. Streicker, S. Altizer, and P. Rohani. Resolving the roles of immunity, pathogenesis, and immigration for rabies persistence in vampire bats. *Proceedings of the National Academy of Sciences of the United States of America*, 110(51):20837–20842, December 2013. doi: 10.1073/pnas.1308817110. URL <https://doi.org/10.1073/pnas.1308817110>.
- James Bradbury, Roy Frostig, Peter Hawkins, Matthew James Johnson, Chris Leary, Dougal Maclaurin, George Necula, Adam Paszke, Jake VanderPlas, Skye Wanderman-Milne, and Qiao Zhang. JAX: Composable transformations of Python+NumPy programs, 2018.
- Carles Bretó. On idiosyncratic stochasticity of financial leverage effects. *Statistics & Probability Letters*, 91:20–26, 2014. doi: 10.1016/j.spl.2014.04.003. URL <https://doi.org/10.1016/j.spl.2014.04.003>.
- Carles Bretó, Daihai He, Edward L. Ionides, and Aaron A. King. Time series analysis via mechanistic models. *The Annals of Applied Statistics*, 3(1):319–348, March 2009. ISSN 1932-6157, 1941-7330. doi: 10.1214/08-AOAS201.
- Carles Bretó, Edward L. Ionides, and Aaron A. King. Panel Data Analysis via Mechanistic Models. *Journal of the American Statistical Association*, 115(531):1178–1188, June 2020. ISSN 0162-1459, 1537-274X. doi: 10.1080/01621459.2019.1604367.
- S. J. Fox, M. Lachmann, M. Tec, R. Pasco, S. Woody, Z. Du, X. Wang, T. A. Ingle, E. Javan, M. Dahan, K. Gaither, M. E. Escott, S. I. Adler, S. C. Johnston, J. G. Scott, and L. A. Meyers. Real-time pandemic surveillance using hospital admissions and mobility data. *Proceedings of the National Academy of Sciences*, 119(7):e2111870119, 2022. doi: 10.1073/pnas.2111870119. URL <https://doi.org/10.1073/pnas.2111870119>.
- Michael B. Giles. Algorithm 955: Approximation of the Inverse Poisson Cumulative Distribution Function. *ACM Transactions on Mathematical Software*, 42(1):1–22, March 2016. ISSN 0098-3500, 1557-7295. doi: 10.1145/2699466. URL <https://dl.acm.org/doi/10.1145/2699466>.
- Michael B. Giles and Casper Beentjes. Approximation of an Inverse of the Incomplete Beta Function. In Kevin Buzzard, Alicia Dickenstein, Bettina Eick, Anton Leykin, and

- Yue Ren, editors, *Mathematical Software – ICMS 2024*, volume 14749, pages 207–214. Springer Nature Switzerland, Cham, 2024. ISBN 978-3-031-64528-0 978-3-031-64529-7. doi: 10.1007/978-3-031-64529-7_22. URL https://link.springer.com/10.1007/978-3-031-64529-7_22. Series Title: Lecture Notes in Computer Science.
- Daihai He, Edward L. Ionides, and Aaron A. King. Plug-and-play inference for disease dynamics: Measles in large and small populations as a case study. *Journal of The Royal Society Interface*, 7(43):271–283, February 2010. ISSN 1742-5689, 1742-5662. doi: 10.1098/rsif.2009.0151.
- E. L. Ionides, C. Breto, and A. A. King. Inference for nonlinear dynamical systems. *Proceedings of the National Academy of Sciences*, 103(49):18438–18443, December 2006. ISSN 0027-8424, 1091-6490. doi: 10.1073/pnas.0603181103.
- Edward L. Ionides, Dao Nguyen, Yves Atchadé, Stilian Stoev, and Aaron A. King. Inference for dynamic and latent variable models via iterated, perturbed Bayes maps. *Proceedings of the National Academy of Sciences*, 112(3):719–724, January 2015. ISSN 0027-8424, 1091-6490. doi: 10.1073/pnas.1410597112.
- Aaron A. King, Dao Nguyen, and Edward L. Ionides. Statistical Inference for Partially Observed Markov Processes via the *R* Package **pomp**. *Journal of Statistical Software*, 69(12), 2016. ISSN 1548-7660. doi: 10.18637/jss.v069.i12.
- Hannah Korevaar, C. Jessica Metcalf, and Bryan T. Grenfell. Structure, space and size: Competing drivers of variation in urban and rural measles transmission. *Journal of The Royal Society Interface*, 17(168):20200010, July 2020. doi: 10.1098/rsif.2020.0010.
- Scott W Linderman, Peter Chang, Giles Harper-Donnelly, Aleyna Kara, Xinglong Li, Gerardo Duran-Martin, and Kevin Murphy. Dynamax: A python package for probabilistic state space modeling with JAX. *Journal of Open Source Software*, 10(108):7069, 2025. doi: 10.21105/joss.07069.
- J. A. Jr. Marino, S. D. Peacor, D. B. Bunnell, H. A. Vanderploeg, S. A. Pothoven, A. K. Elgin, J. R. Bence, J. Jiao, and E. L. Ionides. Evaluating consumptive and nonconsumptive predator effects on prey density using field time-series data. *Ecology*, 100(3):e02583, March 2019. doi: 10.1002/ecy.2583. URL <https://doi.org/10.1002/ecy.2583>.
- Matthew S. Mitchen, Erin Clancey, Corrin McMichael, and Eric T. Lofgren. Estimating sars-cov-2 transmission parameters between coinciding outbreaks in a university population and the surrounding community, Jan 2024. URL <https://doi.org/10.1101/2024.01.10.24301116>.
- Kevin Tan, Giles Hooker, and Edward L. Ionides. Accelerated Inference for Partially Observed Markov Processes using Automatic Differentiation, July 2024.
- N M Temme. Asymptotic Inversion of Incomplete Gamma Functions. April 1992.
- L. Wen, Y. Yin, Q. Li, Z. Peng, and D. He. Modeling the co-circulation of influenza and covid-19 in hong kong, china. *Advances in Continuous and Discrete Models*, 2024

(1):1–9, 2024. doi: 10.1186/s13662-024-03830-7. URL <https://doi.org/10.1186/s13662-024-03830-7>. Article 32.

6 Appendix

[AA: Include imporant numerical results here? Or refer to the quant repo?]